

TẬP ĐOÀN CÔNG NGHIỆP – VIỄN THÔNG QUÂN ĐỘI
CÔNG TY ĐẦU TƯ CÔNG NGHỆ VIETTEL

TÀI LIỆU HỆ THỐNG
HỆ THỐNG QUẢN LÝ TÀI KHOẢN NGÂN HÀNG
BANK-SERVICE

Thực tập sinh: **Phan Thị Thu Hương**
Người hướng dẫn: **Nguyễn Văn Thiệu**
Mã nguồn: **github.com/huong-1406/bank-service**

Hà Nội, tháng 10 năm 2026

LỜI MỞ ĐẦU

Tài liệu này mô tả hệ thống **bank-service**, sản phẩm bài tập trong chương trình thực tập backend tại Công ty Đầu tư Công nghệ Viettel. Hệ thống mô phỏng phần lõi của một ngân hàng gồm quản lý tài khoản, thẻ, số dư và thanh toán, được xây dựng theo kiến trúc nhiều service bằng Spring Boot.

Tài liệu trình bày tổng quan hệ thống, các công nghệ sử dụng, kiến trúc và các thành phần, thiết kế cơ sở dữ liệu, các luồng xử lý chính, cách triển khai và kết quả kiểm thử. Người đọc có thể dùng tài liệu để hiểu hệ thống trước khi đọc mã nguồn, hoặc để tự dựng và chạy lại hệ thống.

Thực tập sinh

Phan Thị Thu Hương

MỤC LỤC

CHƯƠNG 1. TỔNG QUAN HỆ THỐNG.....	1
1.1. Giới thiệu bài toán.....	1
1.2. Mục tiêu.....	1
1.3. Phạm vi hệ thống.....	2
1.4. Tác nhân và chức năng.....	2
1.5. Ràng buộc nghiệp vụ.....	4
1.6. Yêu cầu phi chức năng.....	5
CHƯƠNG 2. CÔNG NGHỆ SỬ DỤNG.....	6
2.1. Java 21 và Spring Boot.....	6
2.2. Mô hình 3 tầng Controller – Service – Repository.....	7
2.3. Spring Security và JWT.....	8
2.3.1. JWT là gì.....	8
2.3.2. Cách dự án sử dụng.....	9
2.4. Redis và Spring Cache.....	10
2.4.1. Redis là gì.....	10
2.4.2. Chiến lược cache-aside.....	10
2.5. ActiveMQ.....	11
2.5.1. Message broker là gì.....	11
2.5.2. Cách dự án sử dụng.....	12
2.6. PostgreSQL và JPA / Hibernate.....	12
2.7. Docker và Docker Compose.....	13
2.8. JUnit 5, Mockito và Postman.....	13
2.9. React.....	14
CHƯƠNG 3. KIẾN TRÚC VÀ CÁC THÀNH PHẦN.....	15
3.1. Kiến trúc tổng thể.....	15
3.2. bank-service.....	16
3.3. payment-service.....	16
3.4. notification-service.....	17
3.5. Giao tiếp giữa các thành phần.....	17
CHƯƠNG 4. THIẾT KẾ CƠ SỞ DỮ LIỆU.....	18
4.1. Mỗi service một database.....	18
4.2. Sơ đồ quan hệ bank_db.....	18
4.3. Chi tiết các bảng.....	19

4.4. Ràng buộc và index.....	20
4.5. Chống trừ tiền hai lần.....	21
4.6. Dữ liệu mẫu.....	21
CHƯƠNG 5. CÁC LƯỠNG XỬ LÝ CHÍNH.....	23
5.1. Đăng nhập và gọi API có token.....	23
5.2. Rút tiền.....	23
5.3. Thanh toán.....	24
5.4. Xử lý lỗi.....	26
CHƯƠNG 6. TRIỂN KHAI VÀ KIỂM THỬ.....	27
6.1. Chạy hệ thống.....	27
6.2. Giao diện.....	27
6.3. Unit test.....	29
6.4. Kiểm thử API bằng Postman.....	30
6.5. Kiểm thử khi một thành phần bị tắt.....	31
CHƯƠNG 7. KẾT LUẬN.....	33
7.1. Kết quả đạt được.....	33
7.2. Hạn chế.....	33
7.3. Hướng phát triển.....	33

DANH MỤC HÌNH

Hình 1.1: Sơ đồ use case của hệ thống.....	3
Hình 2.1: Mô hình 3 tầng trong bank-service.....	7
Hình 2.2: Cấu trúc một JWT trong hệ thống.....	9
Hình 2.3: Luồng đọc dữ liệu theo chiến lược cache-aside.....	10
Hình 2.4: Mô hình producer – queue – consumer trong hệ thống.....	11
Hình 2.5: Dockerfile multi-stage của bank-service.....	13
Hình 3.1: Kiến trúc tổng thể của hệ thống.....	15
Hình 4.1: Sơ đồ thực thể – quan hệ của bank_db.....	18
Hình 5.1: Luồng đăng nhập và gọi API có token.....	23
Hình 5.2: Luồng rút tiền.....	24
Hình 5.3: Luồng thanh toán thành công.....	25
Hình 5.4: Luồng thanh toán khi payment-service không hoạt động.....	25
Hình 6.1: Trang tổng quan: thông tin tài khoản và số dư.....	28
Hình 6.2: Trang thẻ: danh sách thẻ và trạng thái.....	28
Hình 6.3: Trang giao dịch: báo lỗi khi rút quá số dư khả dụng.....	29
Hình 6.4: Kết quả chạy unit test: cả 3 module đều SUCCESS.....	30
Hình 6.5: Kết quả chạy Postman collection bằng Newman: 38 request, 66 kiểm tra, 0 lỗi.....	31
Hình 6.6: payment.queue khi notification-service tắt: 2 message đang chờ, 0 consumer	32

DANH MỤC BẢNG

Bảng 1.1: Phạm vi của hệ thống.....	2
Bảng 1.2: Danh sách API của hệ thống.....	3
Bảng 1.3: Ba ràng buộc nghiệp vụ và cách đảm bảo.....	4
Bảng 1.4: Điều kiện xóa tài khoản và xóa thẻ.....	4
Bảng 1.5: Yêu cầu phi chức năng.....	5
Bảng 2.1: Công nghệ và phiên bản sử dụng.....	6
Bảng 2.2: Trách nhiệm của từng tầng.....	8
Bảng 2.3: Cấu hình bảo mật trong SecurityConfig.....	9
Bảng 2.4: Các thao tác cache trong bank-service.....	11
Bảng 3.1: Các thành phần của hệ thống.....	15
Bảng 3.2: Các package của bank-service.....	16
Bảng 3.3: Các lớp chính của payment-service.....	16
Bảng 3.4: Cách các thành phần giao tiếp.....	17
Bảng 4.1: Ba database của hệ thống.....	18
Bảng 4.2: Bảng account.....	19
Bảng 4.3: Bảng balance.....	19
Bảng 4.4: Bảng card.....	19
Bảng 4.5: Bảng transaction.....	20
Bảng 4.6: Bảng payment (payment_db) và notification (notification_db).....	20
Bảng 4.7: Ràng buộc và index bổ sung.....	21
Bảng 4.8: Hai lệnh rút cùng lúc.....	21
Bảng 4.9: Dữ liệu mẫu.....	22
Bảng 5.1: Số dư qua từng giai đoạn thanh toán 300.000 đ (số dư ban đầu 1.000.000 đ)	26
Bảng 5.2: Các mã lỗi chính.....	26
Bảng 6.1: Địa chỉ truy cập sau khi khởi động.....	27
Bảng 6.2: Số lượng unit test.....	29
Bảng 6.3: Kết quả khi tắt từng thành phần.....	31

THUẬT NGỮ VÀ TỪ VIẾT TẮT

Thuật ngữ	Giải thích
API	Application Programming Interface – giao diện để các chương trình gọi nhau
REST	Kiểu thiết kế API dùng phương thức HTTP (GET, POST, PUT, DELETE) trên các đường dẫn
JWT	JSON Web Token – chuỗi chứng minh danh tính người gọi API
DTO	Data Transfer Object – lớp chứa dữ liệu đi vào hoặc đi ra API
Entity	Lớp Java ánh xạ với một bảng trong cơ sở dữ liệu
ORM / JPA	Kỹ thuật ánh xạ đối tượng sang bảng / chuẩn ORM của Java
Cache	Bộ nhớ đệm, lưu bản sao dữ liệu hay dùng để đọc nhanh hơn
TTL	Time To Live – thời gian sống của một key trong Redis
Message broker	Phần mềm trung gian chuyển tin nhắn giữa các service (ActiveMQ)
Queue	Hàng đợi, nơi tin nhắn nằm chờ đến khi được xử lý
Producer / Consumer	Bên gửi / bên nhận tin nhắn
JMS	Java Message Service – chuẩn Java để làm việc với message broker
Container / Image	Môi trường chạy tách biệt / bản đóng gói để tạo container (Docker)
Transaction	(1) Khối thao tác ghi cùng thành công hoặc cùng hủy; (2) bảng lưu giao dịch nạp, rút, thanh toán
Commit / Rollback	Lưu hẳn các thay đổi xuống database / hủy toàn bộ thay đổi khi có lỗi
PK / FK	Khóa chính (mã riêng của mỗi dòng) / khóa ngoại (cột trỏ tới khóa chính của bảng khác)
UNIQUE / CHECK	Ràng buộc database: không cho trùng giá trị / bắt giá trị phải thỏa điều kiện (ví dụ số dư ≥ 0)
Index	Chỉ mục giúp database tìm dữ liệu nhanh, không phải quét cả bảng
Enum	Kiểu chỉ có vài giá trị cố định, ví dụ ACTIVE / INACTIVE
Mã HTTP	200 thành công · 201 đã tạo · 202 đã nhận · 204 đã xóa · 400 dữ liệu sai · 401 chưa đăng nhập · 404 không tìm thấy · 409 xung đột · 503 dịch vụ tạm ngừng
Healthcheck	Docker định kỳ gọi thử service để biết nó còn chạy tốt

Thuật ngữ	Giải thích
CORS	Cơ chế trình duyệt cho phép trang web gọi API ở địa chỉ khác
SPA	Single Page Application – ứng dụng web một trang (React)

CHƯƠNG 1. TỔNG QUAN HỆ THỐNG

1.1. Giới thiệu bài toán

Trong hoạt động ngân hàng, khách hàng cần quản lý tài khoản, thẻ và số dư của mình một cách an toàn và chính xác. Mỗi thao tác liên quan đến tiền như nạp, rút hay thanh toán đều phải được kiểm tra chặt chẽ để không xảy ra sai lệch số dư.

Hệ thống **bank-service** được xây dựng trong chương trình thực tập tại Công ty Đầu tư Công nghệ Viettel, mô phỏng phần lõi của một hệ thống ngân hàng: quản lý tài khoản khách hàng, quản lý thẻ, quản lý số dư và xử lý thanh toán. Hệ thống gồm 3 service backend viết bằng Spring Boot, một giao diện web viết bằng React, cùng các thành phần hạ tầng PostgreSQL, Redis và ActiveMQ, tất cả được đóng gói và chạy bằng Docker.

Bên cạnh các chức năng nghiệp vụ, dự án áp dụng những kỹ thuật thường gặp trong phát triển backend: phân tầng mã nguồn, xác thực bằng JWT, bộ nhớ đệm Redis, giao tiếp bất đồng bộ qua hàng đợi tin nhắn, triển khai bằng container và kiểm thử tự động.

1.2. Mục tiêu

Hệ thống được xây dựng nhằm đạt các mục tiêu sau:

- Xây dựng đầy đủ API quản lý **tài khoản**, **thẻ** và **số dư**, tổ chức mã nguồn theo 3 tầng Controller – Service – Repository.
- Đảm bảo **3 ràng buộc nghiệp vụ** của đề bài (trình bày ở mục 1.5).
- Lấy mã tài khoản (**accountId**) từ **JWT**, không cho phép người dùng tự truyền qua URL hay nội dung request.
- Dùng **Redis** lưu đệm thông tin tài khoản và số dư với thời gian sống 10 phút, giảm số lần truy vấn cơ sở dữ liệu.
- Xây dựng 2 service riêng là **payment-service** và **notification-service**, giao tiếp với nhau qua hàng đợi **ActiveMQ**.
- Đóng gói toàn bộ hệ thống bằng **Docker**, khởi động bằng một lệnh duy nhất.
- Viết **unit test** (2 test cho mỗi API) và bộ **Postman collection** (mỗi API có ít nhất 1 trường hợp thành công và 1 trường hợp thất bại).
- Xây dựng **giao diện web** để người dùng thao tác trực tiếp với hệ thống.

1.3. Phạm vi hệ thống

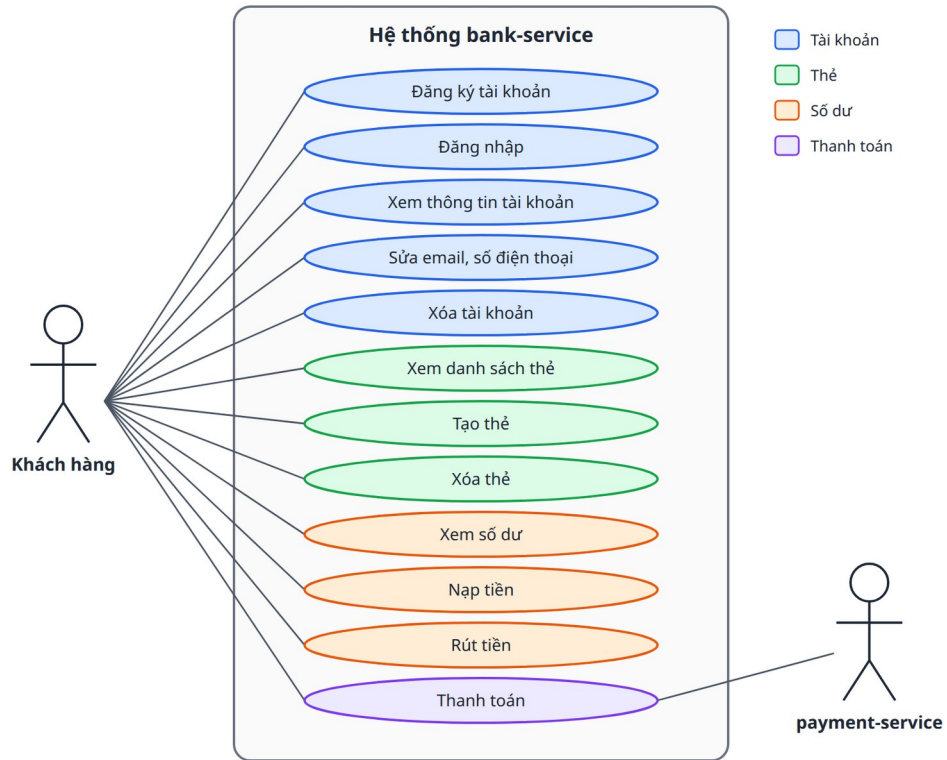
Phạm vi của hệ thống bám sát đề bài. Một số chức năng thường có ở ngân hàng thật nhưng đề bài không yêu cầu sẽ không được xây dựng, để tập trung làm chắc phần được giao. Bảng 1.1 tóm tắt phạm vi này.

Bảng 1.1: Phạm vi của hệ thống

Trong phạm vi	Ngoài phạm vi
Đăng ký, đăng nhập, xem, sửa, xóa tài khoản	Chuyển khoản giữa các tài khoản
Xem, tạo, xóa thẻ (DEBIT / CREDIT)	API kích hoạt hoặc khóa thẻ
Xem số dư, nạp tiền, rút tiền	Xem lịch sử giao dịch
Thanh toán qua payment-service	Kết nối cổng thanh toán thật
Thông báo thanh toán giả lập bằng log	Gửi email hoặc SMS thật
Một loại người dùng: khách hàng	Tài khoản quản trị (admin), phân quyền

1.4. Tác nhân và chức năng

Hệ thống có một tác nhân chính là **khách hàng**, người trực tiếp sử dụng giao diện web hoặc gọi API. Ngoài ra, **payment-service** đóng vai trò tác nhân phụ: nó tham gia vào chức năng thanh toán khi được bank-service gọi sang. Sơ đồ use case ở Hình 1.1 thể hiện các chức năng mà khách hàng sử dụng, được tô màu theo 4 nhóm.



Hình 1.1: Sơ đồ use case của hệ thống

Mỗi chức năng tương ứng với một API của bank-service. Bảng 1.2 liệt kê đầy đủ 12 API công khai và 1 API nội bộ. Các API cần token phải gửi kèm header **Authorization: Bearer <token>**.

Bảng 1.2: Danh sách API của hệ thống

#	Nhóm	Chức năng	Phương thức và đường dẫn	Token
1	Xác thực	Đăng nhập	POST /auth/login	Không
2	Tài khoản	Đăng ký tài khoản	POST /accounts	Không
3	Tài khoản	Xem thông tin tài khoản	GET /accounts/me	Có
4	Tài khoản	Sửa email, số điện thoại	PUT /accounts/me	Có
5	Tài khoản	Xóa tài khoản	DELETE /accounts/me	Có
6	Thẻ	Xem danh sách thẻ	GET /accounts/me/cards	Có
7	Thẻ	Tạo thẻ	POST /accounts/me/cards	Có
8	Thẻ	Xóa thẻ	DELETE /cards/{id}	Có
9	Số dư	Xem số dư	GET /accounts/me/balance	Có
10	Số dư	Nạp tiền	POST /balance/deposit	Có

#	Nhóm	Chức năng	Phương thức và đường dẫn	Token
11	Số dư	Rút tiền	POST /balance/withdraw	Có
12	Thanh toán	Thanh toán	POST /payments	Có
–	Nội bộ	bank-service gọi sang payment-service	POST payment-service:8081/payments	Không

Đường dẫn dùng /me thay cho /{id} vì mã tài khoản được lấy từ token của người đang đăng nhập. Nhờ vậy, người dùng không thể đổi số trên URL để xem tài khoản của người khác. API xóa thẻ là API duy nhất còn mã trên URL; khi đó Service kiểm tra thẻ có thuộc tài khoản đang đăng nhập hay không, nếu không thì trả lỗi 404.

1.5. Ràng buộc nghiệp vụ

Đề bài nhắc lại 3 ràng buộc ở mọi phần bài tập. Đây là các luật bắt buộc, được kiểm tra ở tầng Service và được cơ sở dữ liệu chặn thêm một lớp khi có thể. Bảng 1.3 trình bày từng ràng buộc và cách hệ thống đảm bảo.

Bảng 1.3: Ba ràng buộc nghiệp vụ và cách đảm bảo

Ràng buộc	Cách hệ thống đảm bảo	Lỗi trả về
Chỉ thẻ hợp lệ mới được phép hoạt động	Hàm <code>Card.isValid()</code> yêu cầu thẻ đang ACTIVE và chưa hết hạn. Được gọi trong chức năng rút tiền và thanh toán.	400 CARD_NOT_ACTIVE
Không được tạo thẻ cho tài khoản không tồn tại	<code>CardServiceImpl</code> tìm tài khoản trước khi tạo thẻ. Cột <code>card.account_id</code> là khóa ngoại tới bảng <code>account</code> .	404 ACCOUNT_NOT_FOUND
Không được trừ tiền nếu số dư khả dụng không đủ	So sánh <code>availableBalance</code> với số tiền cần trừ. Cơ sở dữ liệu có thêm ràng buộc CHECK (<code>available_balance >= 0</code>).	400 INSUFFICIENT_BALANCE

Ngoài 3 ràng buộc trên, đề bài còn đặt điều kiện cho thao tác xóa. Các điều kiện này được trình bày ở Bảng 1.4.

Bảng 1.4: Điều kiện xóa tài khoản và xóa thẻ

Thao tác	Điều kiện	Lỗi trả về
Xóa tài khoản	Không còn thẻ liên kết và số dư bằng 0. Kiểm tra cả số dư khả dụng lẫn số dư đang giữ, vì tiền đang giữ vẫn là tiền của khách.	400 ACCOUNT_HAS_CARDS 400 ACCOUNT_BALANCE_NOT_ZERO

Thao tác	Điều kiện	Lỗi trả về
Xóa thẻ	Thẻ không còn giao dịch nào ở trạng thái PENDING (đang chờ xử lý).	400 CARD_HAS_PENDING_TRANSACTION

1.6. Yêu cầu phi chức năng

Bảng 1.5: Yêu cầu phi chức năng

Yêu cầu	Cách đáp ứng
Bảo mật	Mật khẩu lưu dạng băm BCrypt. Mọi API (trừ đăng nhập, đăng ký) yêu cầu JWT có hạn 1 giờ. Truy cập tài nguyên của người khác trả 404 để không lộ thông tin.
Hiệu năng	Cache Redis cho 2 API được gọi nhiều nhất là xem tài khoản và xem số dư, thời gian sống 10 phút.
Tính đúng đắn	Tiền lưu kiểu DECIMAL(19,2) . Các bước ghi của một thao tác (ví dụ trừ tiền và lưu giao dịch) được gom bằng @Transactional : lỗi giữa chừng thì hủy hết, không ghi dở dang. Cột version chống trừ tiền 2 lần khi có 2 lệnh cùng lúc.
Chịu lỗi	Redis tắt thì đọc thẳng database. payment-service tắt thì hoàn tiền và trả 503. notification-service tắt thì message nằm chờ trong hàng đợi.
Triển khai	Toàn bộ 9 container khởi động bằng lệnh docker compose up -d --build .
Kiểm thử	29 unit test (JUnit 5, Mockito) và 38 request Postman, tất cả đều đạt.

CHƯƠNG 2. CÔNG NGHỆ SỬ DỤNG

Chương này giới thiệu các công nghệ được dùng trong hệ thống. Với mỗi công nghệ, tài liệu trình bày ngắn gọn công nghệ đó là gì, lý do lựa chọn và vị trí sử dụng trong dự án. Bảng 2.1 tổng hợp các thành phần chính cùng phiên bản.

Bảng 2.1: Công nghệ và phiên bản sử dụng

Công nghệ	Phiên bản	Vai trò trong hệ thống
Java	21 (LTS)	Ngôn ngữ lập trình cho 3 service backend
Spring Boot	3.4.1	Framework xây dựng các service
Spring Security + jjwt	0.12.6 (jjwt)	Xác thực bằng JWT
Spring Data JPA / Hibernate	theo Spring Boot	Ánh xạ class Java với bảng trong database
PostgreSQL	16	Cơ sở dữ liệu quan hệ, mỗi service một database
Redis	7	Bộ nhớ đệm (cache) cho bank-service
ActiveMQ Classic	5.18.3	Hàng đợi tin nhắn giữa payment-service và notification-service
Docker, Docker Compose	Compose v2	Đóng gói và chạy toàn bộ hệ thống
JUnit 5, Mockito	theo Spring Boot	Viết unit test cho từng Service
Postman, Newman	–	Kiểm thử API
React, Vite, React-Bootstrap	18 / 6 / 2.10	Giao diện web
Lombok	theo Spring Boot	Sinh getter, setter, constructor, giảm mã lặp

2.1. Java 21 và Spring Boot

Java là ngôn ngữ lập trình hướng đối tượng, chạy trên máy ảo JVM. Dự án dùng Java 21, phiên bản được hỗ trợ dài hạn (LTS). Tính năng **record** của Java được dùng để viết các lớp DTO ngắn gọn, ví dụ **LoginRequest**, **BalanceResponse**.

Spring Boot là framework giúp xây dựng ứng dụng Java nhanh chóng. Spring Boot tự cấu hình phần lớn thành phần (auto-configuration), có sẵn web server Tomcat

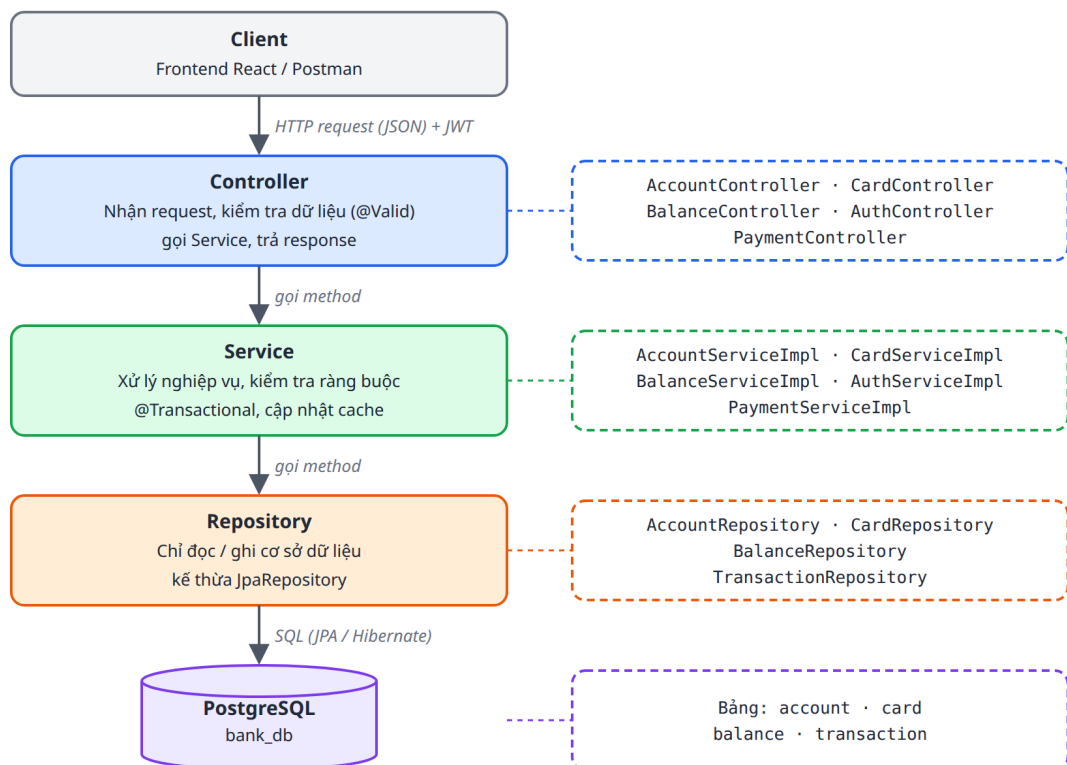
bên trong và quản lý thư viện theo từng gói *starter*. Mỗi service trong dự án là một ứng dụng Spring Boot độc lập.

Khái niệm cốt lõi của Spring là **IoC** (đảo ngược điều khiển) và **Dependency Injection** (tiêm phụ thuộc). Thay vì tự tạo đối tượng bằng `new`, lập trình viên khai báo các lớp là *bean* (`@Service`, `@Repository`, `@RestController`), và Spring tự tạo rồi truyền chúng vào nơi cần dùng qua constructor. Ví dụ `BalanceServiceImpl` nhận sẵn `BalanceRepository`, `CardRepository` mà không phải tự khởi tạo.

Dự án tổ chức theo dạng **Maven multi-module**: một `pom.xml` cha ở thư mục gốc quản lý phiên bản chung, bên dưới là 3 module `bank-service`, `payment-service`, `notification-service`. Mỗi module chỉ khai báo thư viện mình thực sự cần.

2.2. Mô hình 3 tầng Controller – Service – Repository

Đề bài yêu cầu tách mã nguồn thành 3 tầng: phần xử lý vào ra đặt ở **Controller**, phần logic nghiệp vụ đặt ở **Service**, phần tương tác cơ sở dữ liệu đặt ở **Repository**. Mỗi tầng chỉ gọi tầng ngay bên dưới nó, như minh họa ở Hình 2.1.



Hình 2.1: Mô hình 3 tầng trong bank-service

Bảng 2.2: Trách nhiệm của từng tầng

Tầng	Làm gì	Không làm gì
Controller	Nhận request, kiểm tra định dạng dữ liệu bằng @Valid , lấy accountId từ token, gọi Service, trả response với mã HTTP phù hợp.	Không chứa luật nghiệp vụ, không gọi thẳng Repository.
Service	Chứa toàn bộ luật nghiệp vụ (3 ràng buộc, điều kiện xóa), gom các bước ghi bằng @Transactional , cập nhật cache.	Không xử lý HTTP, không viết câu SQL.
Repository	Đọc và ghi cơ sở dữ liệu. Kế thừa JpaRepository nên có sẵn các hàm save , findById , delete .	Không chứa luật nghiệp vụ.

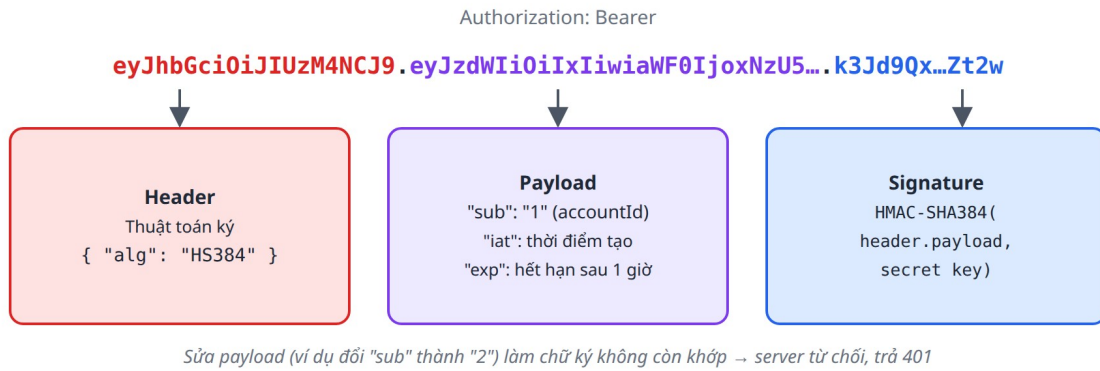
Dữ liệu đi giữa client và Controller dùng các lớp **DTO** (Data Transfer Object) riêng, tách khỏi lớp **Entity** ánh xạ với bảng. Cách tách này giúp không bao giờ trả mật khẩu ra ngoài, và đổi cấu trúc bảng không làm thay đổi API. Các lỗi nghiệp vụ được ném ra dưới dạng **BusinessException** kèm mã lỗi, sau đó **GlobalExceptionHandler** chuyển thành response JSON có cùng một định dạng cho mọi API.

Nhờ phân tầng, tầng Service có thể được kiểm thử độc lập: unit test dùng Mockito tạo Repository giả, không cần chạy database thật.

2.3. Spring Security và JWT

2.3.1. JWT là gì

JWT (JSON Web Token) là một chuỗi ký tự dùng để chứng minh danh tính người gọi API. Chuỗi gồm 3 phần ngăn cách bởi dấu chấm: **Header** ghi thuật toán ký, **Payload** chứa thông tin (trong dự án là mã tài khoản), và **Signature** là chữ ký được tạo từ hai phần trên cùng một khóa bí mật (secret key) chỉ server biết. Cấu trúc này được minh họa ở Hình 2.2.



Hình 2.2: Cấu trúc một JWT trong hệ thống

Vì chữ ký phụ thuộc vào nội dung, chỉ cần sửa một ký tự trong payload thì chữ ký sẽ không còn khớp và server từ chối token. Người dùng có thể đọc được payload nhưng không thể giả mạo. Ngoài ra, JWT là cơ chế *stateless*: server không cần lưu phiên đăng nhập, chỉ cần kiểm tra chữ ký và thời hạn của token trong mỗi request.

2.3.2. Cách dự án sử dụng

- Đăng nhập:** `AuthServiceImpl` tìm tài khoản theo email, so mật khẩu bằng BCrypt. Đúng thì `JwtUtil` tạo token, ghi `accountId` vào trường `sub`, hạn 1 giờ, ký bằng thuật toán HS384.
- Gọi API:** client gửi token trong header `Authorization: Bearer <token>`.
- Kiểm tra token:** `JwtAuthenticationFilter` chạy trước mọi Controller, đọc header, kiểm tra chữ ký và thời hạn, rồi lưu `accountId` vào `SecurityContextHolder`. Token sai hoặc hết hạn thì không lưu gì, API trả 401.
- Lấy accountId:** Controller gọi `SecurityUtil.currentAccountId()` để biết ai đang gọi API.

Bảng 2.3: Cấu hình bảo mật trong SecurityConfig

Mục	Cấu hình
API không cần token	<code>POST /auth/login</code> , <code>POST /accounts</code> , <code>/actuator/health</code>
API còn lại	Bắt buộc có token hợp lệ
Phiên đăng nhập	STATELESS , không tạo session phía server
CSRF	Tắt. CSRF là kiểu tấn công lợi dụng cookie đăng nhập có sẵn trong trình duyệt; hệ thống không dùng cookie mà gửi token qua header nên không cần bật
CORS	Cho phép frontend tại <code>http://localhost:3000</code> gọi API

Mục	Cấu hình
Lưu mật khẩu	Băm bằng <code>BCryptPasswordEncoder</code> , không lưu mật khẩu gốc

Lưu ý bảo mật: Sai email và sai mật khẩu trả về cùng một lỗi `INVALID_CREDENTIALS`. Nếu báo riêng "email không tồn tại", kẻ xấu có thể dò ra email nào đã đăng ký.

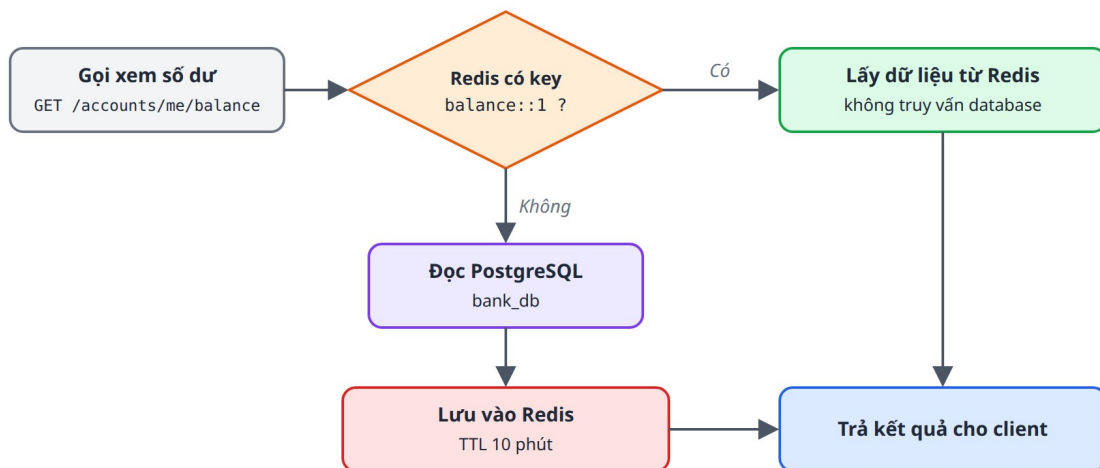
2.4. Redis và Spring Cache

2.4.1. Redis là gì

Redis là cơ sở dữ liệu lưu trong bộ nhớ RAM theo dạng khóa – giá trị (key – value). Vì đọc từ RAM nên Redis nhanh hơn nhiều so với truy vấn PostgreSQL trên ổ đĩa. Redis thường được dùng làm **bộ nhớ đệm (cache)**: lưu bản sao của dữ liệu hay được đọc, để giảm tải cho database chính. Mỗi key có thể đặt **TTL** (Time To Live, thời gian sống); hết thời gian này Redis tự xóa key.

2.4.2. Chiến lược cache-aside

Dự án áp dụng chiến lược **cache-aside** đúng như đề bài mô tả: khi đọc dữ liệu, hệ thống kiểm tra Redis trước; có thì trả về ngay, không có thì đọc database rồi lưu kết quả vào Redis cho lần sau. Luồng này được thể hiện ở Hình 2.3.



Hình 2.3: Luồng đọc dữ liệu theo chiến lược cache-aside

Spring Cache cho phép làm việc này bằng annotation, không cần viết code gọi Redis thủ công. Bảng 2.4 tóm tắt cách dự án dùng cache.

Bảng 2.4: Các thao tác cache trong bank-service

Thao tác	Cách làm	Key Redis
Xem tài khoản	@Cacheable: có trong Redis thì trả luôn, không thì đọc DB rồi lưu	account::{id}
Xem số dư	@Cacheable, tương tự	balance::{id}
Sửa tài khoản	@CacheEvict: xóa key, lần đọc sau lấy dữ liệu mới từ DB	account::{id}
Xóa tài khoản	@CacheEvict cả 2 key	cả 2 key
Nạp, rút, thanh toán	Ghi đè số dư mới vào Redis ngay sau khi lưu DB thành công	balance::{id}

{id} là mã tài khoản, ví dụ tài khoản số 1 có key **account::1** và **balance::1**.

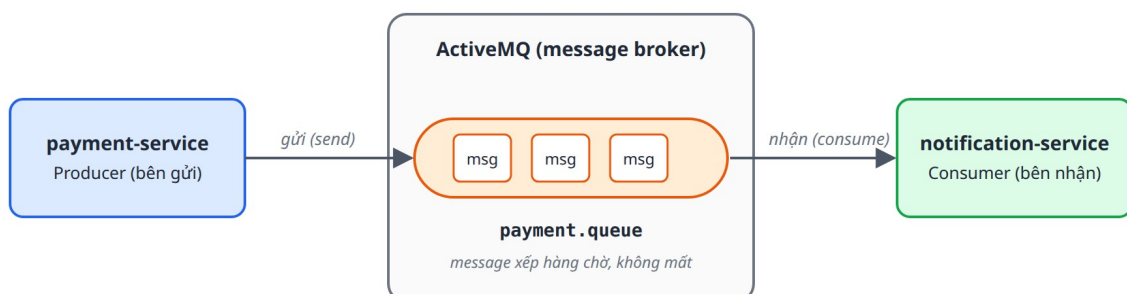
Thời gian sống được đặt là 10 phút tại **application.yml** (**spring.cache.redis.time-to-live: 600000**) và áp dụng cho mọi key trong **RedisConfig**. Dữ liệu trong Redis lưu dạng JSON nên có thể đọc trực tiếp bằng **redis-cli** khi cần kiểm tra.

Ghi chú: Redis chỉ là bản sao. Nếu Redis gặp sự cố, lớp **ResilientCache** bắt lỗi, ghi cảnh báo vào log và đọc thẳng từ database. Hệ thống vẫn hoạt động, chỉ chậm hơn.

2.5. ActiveMQ

2.5.1. Message broker là gì

ActiveMQ là một *message broker*: phần mềm trung gian nhận tin nhắn (message) từ một service và chuyển cho service khác. Bên gửi gọi là **producer**, bên nhận gọi là **consumer**. Tin nhắn được xếp vào một **hàng đợi (queue)**; mỗi message chỉ được một consumer nhận và xử lý. Mô hình này được minh họa ở Hình 2.4.



Hình 2.4: Mô hình producer – queue – consumer trong hệ thống

Điểm khác biệt quan trọng so với gọi HTTP trực tiếp là **giao tiếp bất đồng bộ**: producer gửi message vào hàng đợi là xong việc, không cần chờ consumer xử lý. Nếu consumer đang tắt, message vẫn nằm chờ trong hàng đợi và được xử lý khi consumer chạy lại, không bị mất.

2.5.2. Cách dự án sử dụng

- **payment-service** (producer) nhận yêu cầu thanh toán từ bank-service qua HTTP, rồi gửi message vào hàng đợi **payment.queue** bằng **JmsTemplate** (lớp có sẵn của Spring dùng để gửi message).
- Message gồm 4 trường theo đúng đề bài: **paymentId**, **accountId**, **amount**, **currency**, gửi dưới dạng JSON.
- **notification-service** (consumer) dùng **@JmsListener** nghe hàng đợi **payment.queue**, nhận message và ghi log **Payment confirmed for paymentId:**
- Cả hai phía đều **chống xử lý trùng** theo **paymentId**, vì ActiveMQ có thể giao lại một message khi consumer gặp sự cố giữa chừng.

2.6. PostgreSQL và JPA / Hibernate

PostgreSQL là hệ quản trị cơ sở dữ liệu quan hệ mã nguồn mở, hỗ trợ đầy đủ khóa ngoại, ràng buộc **CHECK** và transaction (khối ghi cùng thành công hoặc cùng hủy). Dự án dùng PostgreSQL 16, mỗi service có một database riêng chạy trong container riêng.

JPA là chuẩn của Java để ánh xạ đối tượng sang bảng (ORM), **Hibernate** là thư viện hiện thực chuẩn này. Mỗi lớp được đánh dấu **@Entity** tương ứng một bảng, mỗi thuộc tính tương ứng một cột. Spring Data JPA sinh sẵn câu truy vấn từ tên hàm, ví dụ **existsByCardIdAndStatus** tự sinh câu lệnh kiểm tra thẻ còn giao dịch ở trạng thái cho trước hay không.

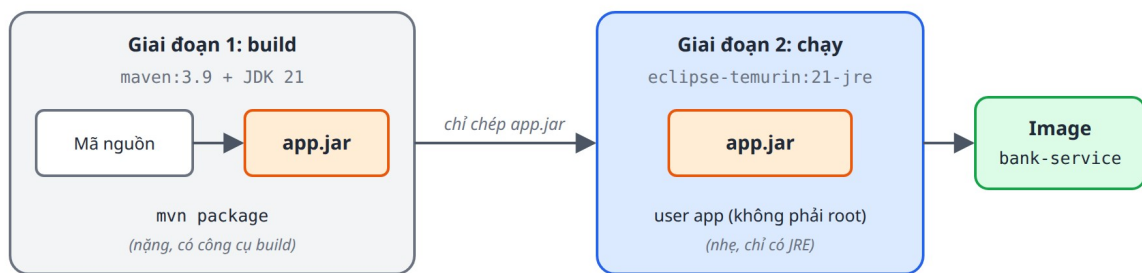
Bảng được Hibernate tự tạo từ entity (**ddl-auto: update**). Những phần Hibernate không tự tạo được như index và ràng buộc **CHECK** được viết trong file **schema-extra.sql**; dữ liệu mẫu nằm trong **data.sql**. Hai file này tự chạy mỗi khi ứng dụng khởi động.

Annotation **@Transactional** gom nhiều thao tác ghi thành một khối: tất cả cùng thành công, hoặc cùng bị hủy nếu có lỗi. Ví dụ khi rút tiền, việc trừ số dư và lưu giao dịch luôn đi cùng nhau. Chi tiết thiết kế cơ sở dữ liệu được trình bày ở Chương 4.

2.7. Docker và Docker Compose

Docker đóng gói ứng dụng cùng toàn bộ môi trường cần thiết vào một **image**. Khi chạy, image tạo ra **container**: một môi trường tách biệt, có mạng và cổng riêng. Nhờ vậy hệ thống chạy giống nhau trên mọi máy, không phụ thuộc máy đó đã cài Java hay PostgreSQL hay chưa.

Mỗi service backend có một **Dockerfile** viết theo kiểu **multi-stage** (Hình 2.5): giai đoạn đầu dùng image Maven có JDK để biên dịch ra file **app.jar**, giai đoạn sau chỉ chép file này sang image JRE gọn nhẹ và chạy bằng người dùng không có quyền root. Image cuối cùng nhỏ hơn và an toàn hơn vì không chứa công cụ build.



Hình 2.5: Dockerfile multi-stage của bank-service

Docker Compose khai báo nhiều container trong một file **docker-compose.yml** và khởi động tất cả bằng một lệnh. Hệ thống gồm 9 container; mỗi container có healthcheck (Docker định kỳ gọi thử, ví dụ **/actuator/health**, để biết container còn chạy tốt), tự khởi động lại khi gặp lỗi và được giới hạn bộ nhớ. Kiến trúc container được trình bày chi tiết ở Chương 3.

2.8. JUnit 5, Mockito và Postman

JUnit 5 là framework viết test cho Java. **Mockito** tạo đối tượng giả (mock) thay cho Repository, giúp kiểm thử riêng logic của tầng Service mà không cần database. Mỗi API có ít nhất 2 test: một trường hợp thành công và một trường hợp thất bại.

Postman là công cụ gửi request để kiểm thử API. Dự án có một bộ Postman collection gồm 38 request, tự lưu token sau khi đăng nhập và tự dọn dữ liệu sau khi chạy. **Newman** là công cụ chạy collection này bằng dòng lệnh. Kết quả kiểm thử được trình bày ở Chương 6.

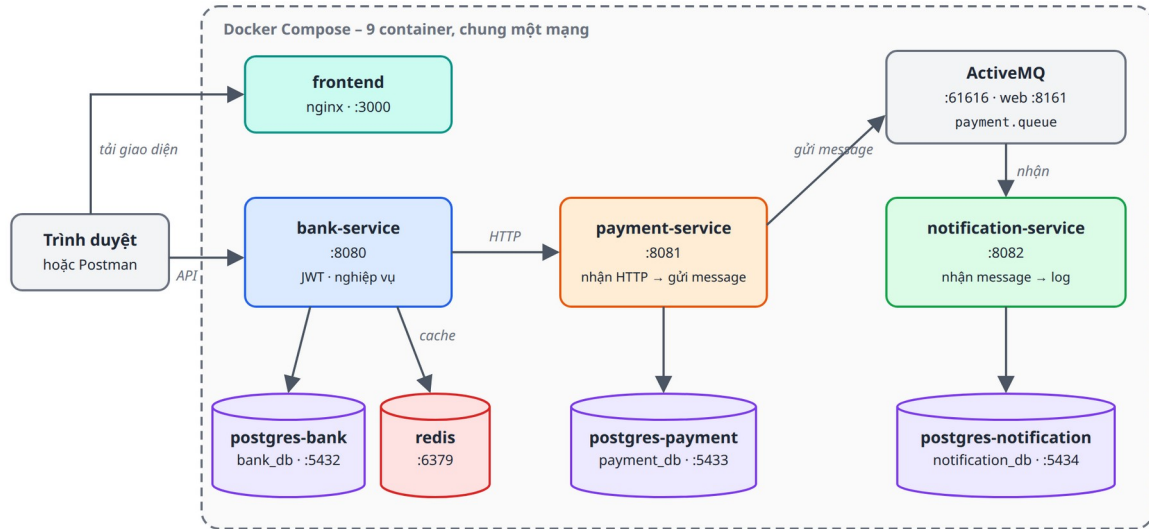
2.9. React

React là thư viện JavaScript xây dựng giao diện theo dạng component. Giao diện của hệ thống là một ứng dụng trang đơn (SPA), dùng **Vite** để build, **React Router** để chuyển trang, **Axios** để gọi API và **React-Bootstrap** để dựng giao diện. Sau khi build, giao diện chỉ là các file tĩnh, được web server **nginx** phục vụ trong một container riêng ở cổng 3000.

CHƯƠNG 3. KIẾN TRÚC VÀ CÁC THÀNH PHẦN

3.1. Kiến trúc tổng thể

Hệ thống gồm 9 container chạy chung một mạng Docker (Hình 3.1). Người dùng tải giao diện từ container frontend, sau đó trình duyệt gọi thẳng API của bank-service. Chỉ bank-service nhận request từ bên ngoài; payment-service và notification-service chỉ được gọi nội bộ.



Hình 3.1: Kiến trúc tổng thể của hệ thống

Bảng 3.1: Các thành phần của hệ thống

Container	Cổng	Vai trò
frontend	3000	nginx phục vụ giao diện React đã build
bank-service	8080	Toàn bộ nghiệp vụ: tài khoản, thẻ, số dư, thanh toán; xác thực JWT
payment-service	8081	Nhận yêu cầu thanh toán từ bank-service, gửi message vào ActiveMQ
notification-service	8082	Nhận message từ ActiveMQ, lưu và ghi log thông báo
postgres-bank	5432	Database bank_db của bank-service
postgres-payment	5433	Database payment_db của payment-service
postgres-notification	5434	Database notification_db của notification-service
redis	6379	Cache tài khoản và số dư
activemq	61616, 8161	Hàng đợi payment.queue ; cổng 8161 là trang web có sẵn của ActiveMQ để xem hàng đợi

Ghi chú: Cổng trong bảng là cổng mở ra máy chủ. Bên trong Docker, các container gọi nhau bằng tên container, ví dụ bank-service gọi <http://payment-service:8081>.

3.2. bank-service

Service chính, chứa toàn bộ luật nghiệp vụ. Mã nguồn chia theo package như Bảng 3.2.

Bảng 3.2: Các package của bank-service

Package	Nội dung
controller	AuthController, AccountController, CardController, BalanceController, PaymentController
service, service.impl	Interface và lớp xử lý nghiệp vụ tương ứng từng controller
repository	AccountRepository, CardRepository, BalanceRepository, TransactionRepository
entity	Account, Card, Balance, Transaction và 4 enum trạng thái, loại
dto	Lớp request và response của API
security	JwtUtil, JwtAuthenticationFilter, SecurityConfig, SecurityUtil
config	RedisConfig, ResilientCache (bỏ qua cache khi Redis lỗi)
client	PaymentClient: gọi HTTP sang payment-service, timeout 3 giây
exception	ErrorCode, BusinessException, GlobalExceptionHandler

3.3. payment-service

Service nhỏ, không kiểm tra nghiệp vụ vì bank-service đã kiểm tra trước khi gọi sang. Khi nhận **POST /payments**, service lưu yêu cầu vào **payment_db** rồi gửi message vào **payment.queue**, sau đó trả 202. Nếu cùng một **paymentId** được gửi lại, service bỏ qua để không tạo message thứ hai.

Bảng 3.3: Các lớp chính của payment-service

Lớp	Vai trò
PaymentController	Nhận POST /payments , trả 202 (Accepted: đã nhận yêu cầu)
PaymentServiceImpl	Chống trùng, lưu bảng payment , gửi message bằng JmsTemplate của Spring

Lớp	Vai trò
PaymentMessage	Nội dung message: paymentId , accountId , amount , currency
JmsConfig	Tên hàng đợi, chuyển message sang JSON

3.4. notification-service

Service không có Controller vì không nhận HTTP. Đầu vào là **PaymentListener** dùng **@JmsListener** nghe **payment.queue**. Mỗi message nhận được sẽ được **NotificationServiceImpl** kiểm tra trùng, lưu vào bảng **notification** và ghi log **Payment confirmed for paymentId: ...**

3.5. Giao tiếp giữa các thành phần

Bảng 3.4: Cách các thành phần giao tiếp

Từ → Đến	Cách giao tiếp	Kiểu
Trình duyệt → bank-service	HTTP REST, JSON, header JWT	Đồng bộ
bank-service → PostgreSQL, Redis	JPA / Spring Cache	Đồng bộ
bank-service → payment-service	HTTP POST /payments (RestClient)	Đồng bộ
payment-service → notification-service	Qua ActiveMQ, hàng đợi payment.queue	Bất đồng bộ

bank-service cần biết ngay payment-service có nhận yêu cầu hay không để trừ hân tiền hoặc hoàn tiền, nên dùng HTTP. Ngược lại, việc gửi thông báo không cần xong ngay, nên đi qua hàng đợi: notification-service tắt thì thanh toán vẫn thành công, thông báo chỉ đến muộn hơn.

CHƯƠNG 4. THIẾT KẾ CƠ SỞ DỮ LIỆU

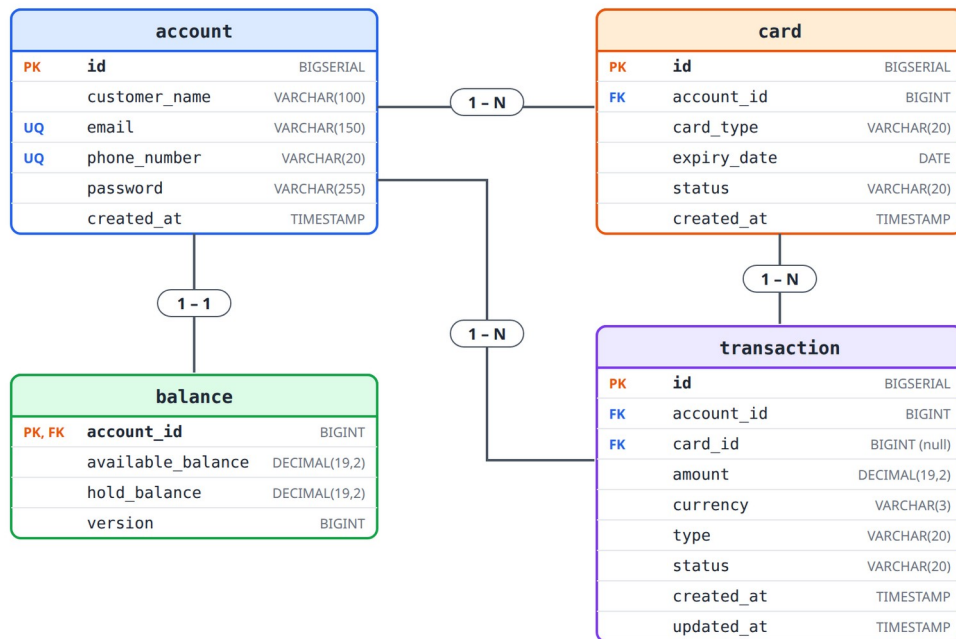
4.1. Mỗi service một database

Mỗi service có database riêng trong container riêng và chỉ kết nối tới database của mình. Một database gặp sự cố không làm hỏng service khác. Vì nằm ở database khác nhau, các cột liên kết giữa service (ví dụ `payment.transaction_id`) không phải khóa ngoại.

Bảng 4.1: Ba database của hệ thống

Database	Service	Bảng
bank_db	bank-service	account, balance, card, transaction
payment_db	payment-service	payment
notification_db	notification-service	notification

4.2. Sơ đồ quan hệ bank_db



1 tài khoản có đúng 1 số dư, nhiều thẻ và nhiều giao dịch; 1 thẻ có nhiều giao dịch (nạp tiền không gắn thẻ)

Hình 4.1: Sơ đồ thực thể – quan hệ của bank_db

Bảng **balance** dùng **account_id** vừa làm khóa chính vừa làm khóa ngoại, nên mỗi tài khoản chỉ có đúng một dòng số dư. Bảng **transaction** không có trong danh sách thực

thể của đề bài nhưng bắt buộc phải có, vì đề yêu cầu chặn xóa thẻ còn giao dịch đang chờ xử lý.

4.3. Chi tiết các bảng

Bảng 4.2: Bảng account

Cột	Kiểu	Ràng buộc	Ý nghĩa
id	BIGSERIAL	PK	Mã tài khoản
customer_name	VARCHAR(100)	NOT NULL	Tên khách hàng
email	VARCHAR(150)	UNIQUE	Dùng để đăng nhập
phone_number	VARCHAR(20)	UNIQUE	Số điện thoại
password	VARCHAR(255)	NOT NULL	Mật khẩu đã băm BCrypt
created_at	TIMESTAMP	NOT NULL	Thời điểm tạo

Bảng 4.3: Bảng balance

Cột	Kiểu	Ràng buộc	Ý nghĩa
account_id	BIGINT	PK, FK	Tài khoản sở hữu; xóa tài khoản thì xóa theo
available_balance	DECIMAL(19,2)	≥ 0	Số dư khả dụng, được phép tiêu
hold_balance	DECIMAL(19,2)	≥ 0	Số dư đang giữ cho thanh toán chưa xong
version	BIGINT	NOT NULL	Chống ghi đè khi 2 lệnh cùng lúc (mục 4.5)

Bảng 4.4: Bảng card

Cột	Kiểu	Ràng buộc	Ý nghĩa
id	BIGSERIAL	PK	Mã thẻ
account_id	BIGINT	FK, RESTRICT	Tài khoản sở hữu; còn thẻ thì không xóa được tài khoản
card_type	VARCHAR(20)	NOT NULL	DEBIT hoặc CREDIT
expiry_date	DATE	NOT NULL	Ngày hết hạn
status	VARCHAR(20)	NOT NULL	ACTIVE đang dùng được, INACTIVE bị tắt; mặc định ACTIVE
created_at	TIMESTAMP	NOT NULL	Thời điểm tạo

Bảng 4.5: Bảng transaction

Cột	Kiểu	Ràng buộc	Ý nghĩa
id	BIGSERIAL	PK	Mã giao dịch, cũng là paymentId gửi sang payment-service
account_id	BIGINT	FK, CASCADE	Tài khoản thực hiện; xóa tài khoản thì giao dịch xóa theo
card_id	BIGINT	FK, CASCADE, được NULL	Thẻ dùng; nạp tiền thì để trống; xóa thẻ thì giao dịch xóa theo
amount	DECIMAL(19,2)	> 0	Số tiền
currency	VARCHAR(3)	NOT NULL	Mặc định VND
type	VARCHAR(20)	NOT NULL	DEPOSIT , WITHDRAW hoặc PAYMENT
status	VARCHAR(20)	NOT NULL	PENDING đang chờ, COMPLETED xong, FAILED thất bại (đã hoàn tiền)
created_at, updated_at	TIMESTAMP		Thời điểm tạo, cập nhật

Bảng 4.6: Bảng payment (payment_db) và notification (notification_db)

Bảng	Cột chính	Chống trùng
payment	id, transaction_id, account_id, amount, currency, status (RECEIVED → SENT), created_at	transaction_id UNIQUE
notification	id, payment_id, account_id, amount, currency, message, created_at	payment_id UNIQUE

Quy ước chung: tiền lưu kiểu **DECIMAL(19,2)** để không sai số như **FLOAT**; enum lưu dạng chuỗi để đọc database là hiểu ngay.

4.4. Ràng buộc và index

Hibernate tự tạo bảng, khóa chính, khóa ngoại và UNIQUE từ entity. Phần còn lại nằm trong **schema-extra.sql** (Bảng 4.7).

Bảng 4.7: Ràng buộc và index bổ sung

Tên	Nội dung	Mục đích
chk_balance_available_non_negative	available_balance >= 0	Số dư không âm
chk_balance_hold_non_negative	hold_balance >= 0	Tiền giữ không âm
chk_txn_amount_positive	amount > 0	Số tiền giao dịch dương
chk_txn_card_required	type = 'DEPOSIT' OR card_id IS NOT NULL	Rút và thanh toán bắt buộc có thẻ
idx_card_account	card(account_id)	Liệt kê thẻ nhanh
idx_txn_account	transaction(account_id)	Tìm giao dịch theo tài khoản
idx_txn_card_status	transaction(card_id, status)	Kiểm tra thẻ còn giao dịch PENDING

4.5. Chống trừ tiền hai lần

Nếu 2 lệnh rút tiền của cùng một tài khoản chạy cùng lúc, cả hai đều đọc thấy đủ tiền và có thể cùng trừ. Hệ thống chặn bằng cột **version** (annotation **@Version** trong entity **Balance**). Mỗi lần ghi, Hibernate chỉ cập nhật dòng có **version** đúng bằng lúc đọc, rồi tăng **version** lên 1. Bảng 4.8 minh họa với số dư 1.000.000 đ và 2 lệnh rút 600.000 đ.

Bảng 4.8: Hai lệnh rút cùng lúc

Bước	Lệnh 1	Lệnh 2
1	Đọc: 1.000.000, version 5	Đọc: 1.000.000, version 5
2	Ghi 400.000, version thành 6 → thành công	
3		Tìm dòng version 5, không còn → lỗi 409 CONCURRENT_UPDATE , không trừ tiền

4.6. Dữ liệu mẫu

File **data.sql** tạo sẵn 4 tài khoản để kiểm thử, mật khẩu chung **Test@1234**.

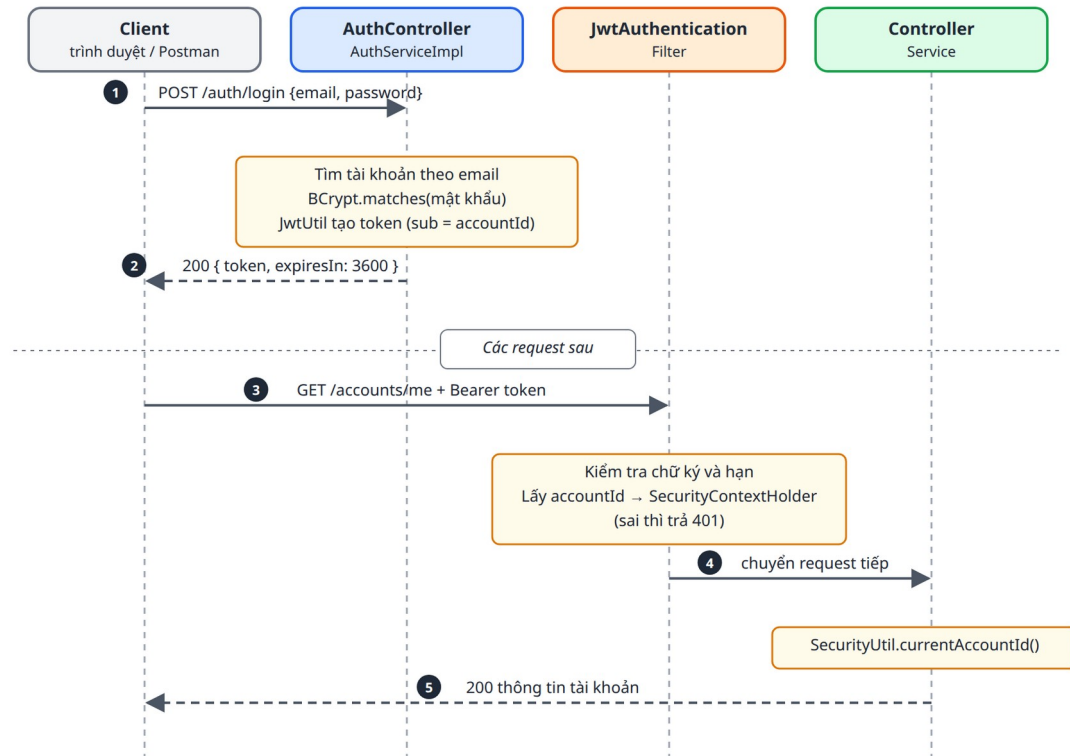
Bảng 4.9: Dữ liệu mẫu

Email	Số dư	Thẻ	Dùng để kiểm thử
a@test.com	1.000.000	ACTIVE, hạn 31/12/2030	Các trường hợp thành công
b@test.com	0	Không có	Xóa tài khoản thành công
c@test.com	500.000	INACTIVE	Rút tiền bị chặn vì thẻ tắt
d@test.com	500.000	ACTIVE, hạn 31/01/2020	Rút tiền bị chặn vì thẻ hết hạn

CHƯƠNG 5. CÁC LƯỒNG XỬ LÝ CHÍNH

5.1. Đăng nhập và gọi API có token

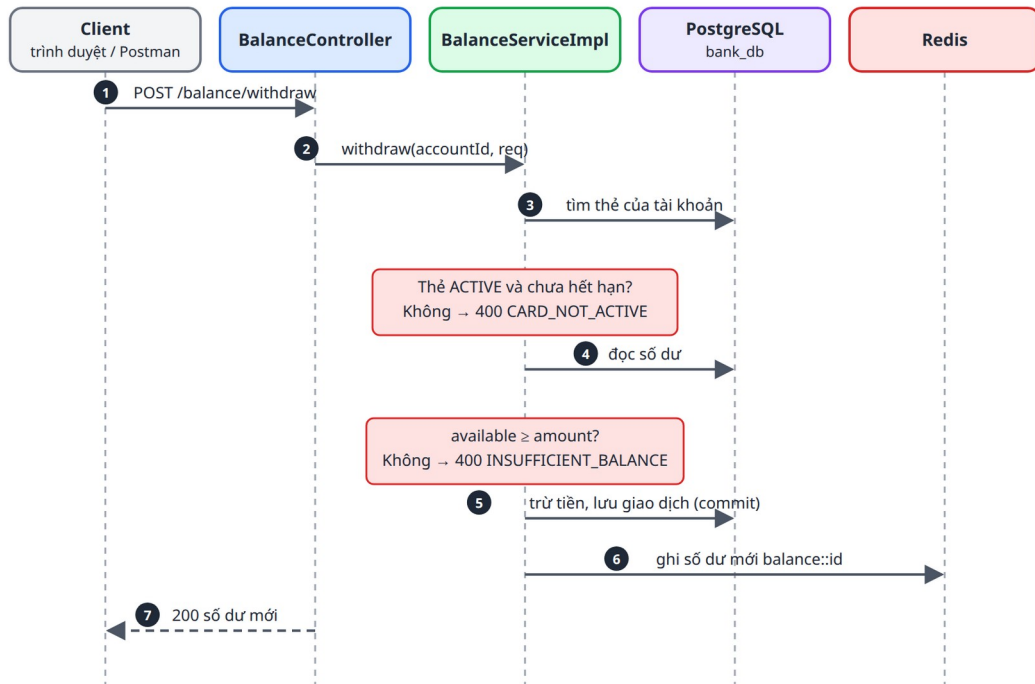
Đăng nhập trả về token chứa **accountId**. Các request sau gửi token trong header, **JwtAuthenticationFilter** kiểm tra rồi đưa **accountId** cho Controller (Hình 5.1).



Hình 5.1: Luồng đăng nhập và gọi API có token

5.2. Rút tiền

Service kiểm tra thẻ trước, số dư sau; vi phạm điều nào thì dừng và trả lỗi ngay. Hợp lệ thì trừ tiền, lưu giao dịch rồi **commit** (lưu hẳn xuống database) trong cùng một **@Transactional**; sau đó mới ghi số dư mới vào Redis (Hình 5.2). Nạp tiền đi theo luồng tương tự nhưng không cần thẻ và không kiểm tra số dư.

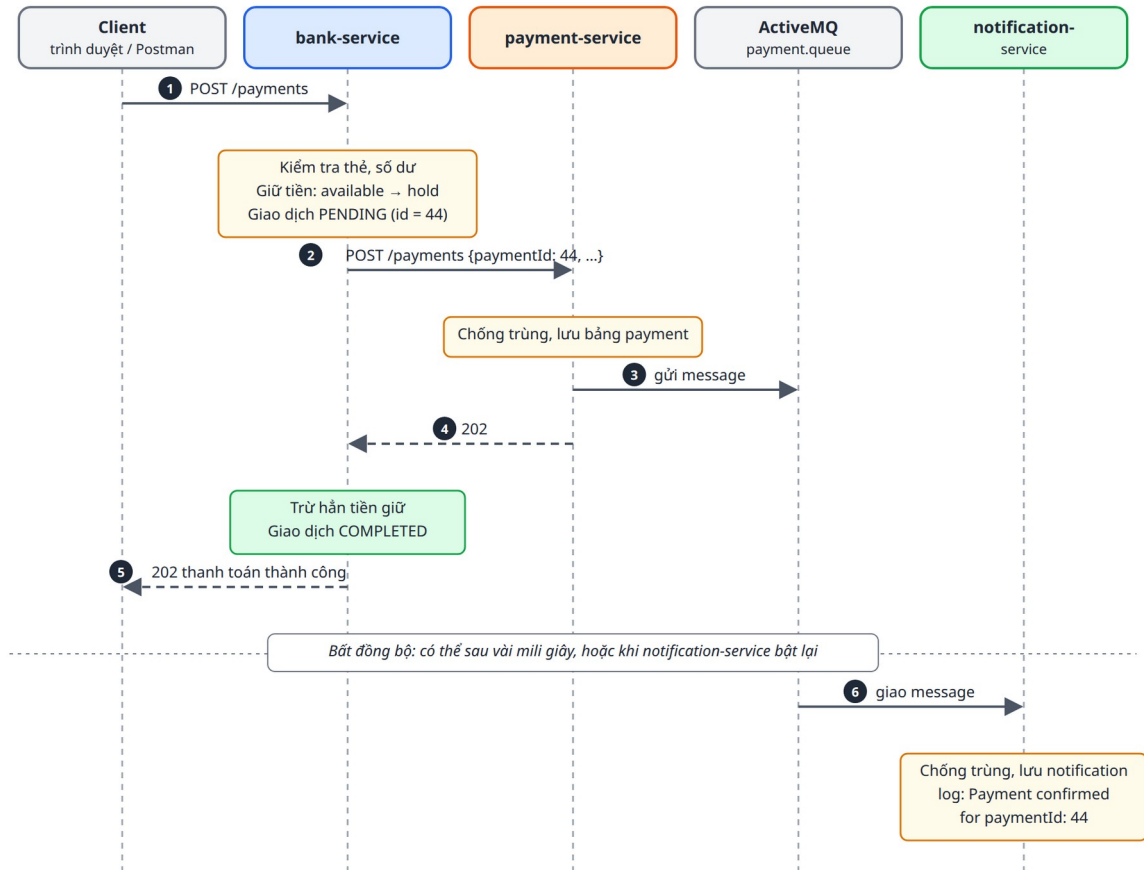


Hình 5.2: Luồng rút tiền

5.3. Thanh toán

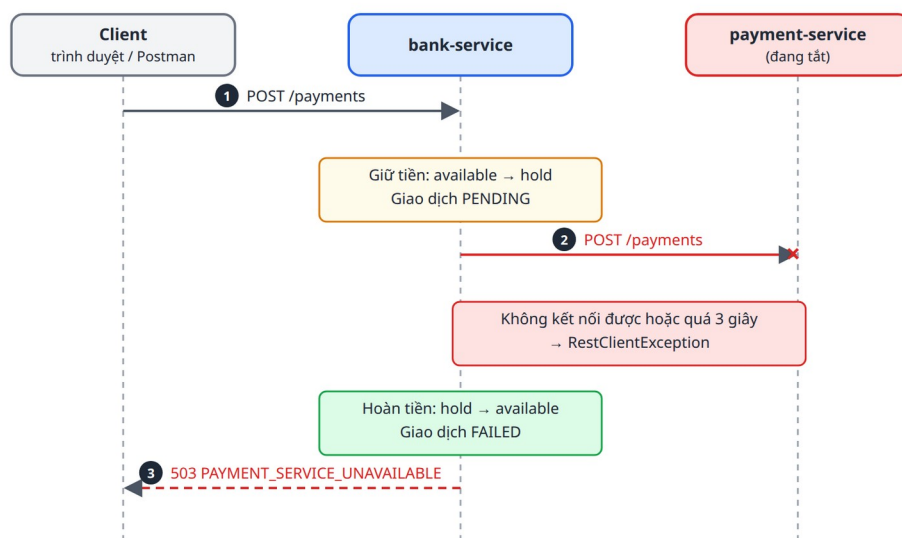
Thanh toán đi qua cả 3 service (Hình 5.3). bank-service giữ tiền trước khi gọi payment-service để trong lúc chờ, số tiền này không bị dùng cho giao dịch khác. Phần thông báo chạy bất đồng bộ nên không làm chậm phản hồi cho người dùng.

- **Giữ tiền (available → hold):** chuyển số tiền từ số dư khả dụng sang số dư đang giữ. Khách không tiêu được số này nữa, nhưng tiền chưa mất hẳn.
- **202 (Accepted):** mã HTTP nghĩa là "đã nhận". Dùng 202 thay cho 200 vì lúc trả kết quả, việc thông báo vẫn đang chạy phía sau. Tiền thì đã trừ xong, giao dịch đã **COMPLETED**.



Hình 5.3: Luồng thanh toán thành công

Nếu payment-service không phản hồi (tắt, hoặc quá 3 giây), **PaymentClient** báo lỗi **RestClientException**. bank-service bắt lỗi này, hoàn lại tiền đang giữ và trả 503 (Hình 5.4). Số dư của khách không thay đổi.



Hình 5.4: Luồng thanh toán khi payment-service không hoạt động

Bảng 5.1: Số dư qua từng giai đoạn thanh toán 300.000 đ (số dư ban đầu 1.000.000 đ)

Giai đoạn	Khả dụng	Đang giữ	Giao dịch
Giữ tiền	700.000	300.000	PENDING
payment-service nhận (202)	700.000	0	COMPLETED
payment-service lỗi	1.000.000	0	FAILED

5.4. Xử lý lỗi

Mọi lỗi trả về cùng một định dạng JSON do **GlobalExceptionHandler** tạo, nên frontend chỉ cần một chỗ để hiển thị lỗi.

```
{
  "timestamp": "2026-10-05T10:30:00",
  "status": 400,
  "code": "INSUFFICIENT_BALANCE",
  "message": "Số dư khả dụng không đủ",
  "path": "/balance/withdraw"
}
```

Bảng 5.2: Các mã lỗi chính

HTTP	Mã lỗi	Khi nào
400	VALIDATION_FAILED	Dữ liệu sai định dạng (kèm fieldErrors)
400	INSUFFICIENT_BALANCE	Số dư khả dụng không đủ
400	CARD_NOT_ACTIVE	Thẻ INACTIVE hoặc đã hết hạn
400	CARD_HAS_PENDING_TRANSACTION	Xóa thẻ còn giao dịch đang chờ
400	ACCOUNT_HAS_CARDS, ACCOUNT_BALANCE_NOT_ZERO	Xóa tài khoản còn thẻ hoặc còn tiền
401	INVALID_CREDENTIALS, UNAUTHORIZED	Sai email/mật khẩu; thiếu hoặc sai token
404	ACCOUNT_NOT_FOUND, CARD_NOT_FOUND	Không tìm thấy, hoặc không phải của mình
409	EMAIL_ALREADY_EXISTS, PHONE_ALREADY_EXISTS	Email hoặc số điện thoại đã được dùng
409	CONCURRENT_UPDATE	Hai lệnh cùng sửa số dư (mục 4.5)
503	PAYMENT_SERVICE_UNAVAILABLE	payment-service không phản hồi, đã hoàn tiền

CHƯƠNG 6. TRIỂN KHAI VÀ KIỂM THỬ

6.1. Chạy hệ thống

Máy chỉ cần cài Docker. Tại thư mục gốc của dự án:

```
# Build và khởi động 9 container
docker compose up -d --build

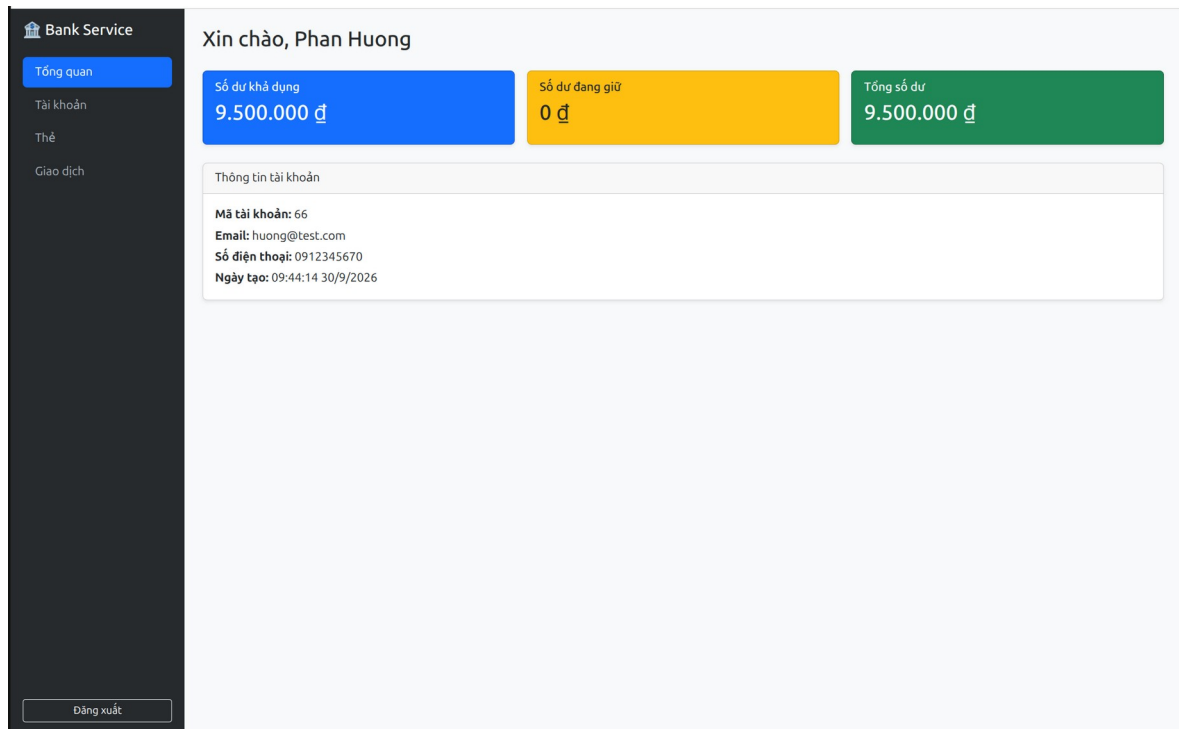
# Kiểm tra trạng thái: cột STATUS phải là "healthy" (đã chạy tốt)
docker compose ps
```

Bảng 6.1: Địa chỉ truy cập sau khi khởi động

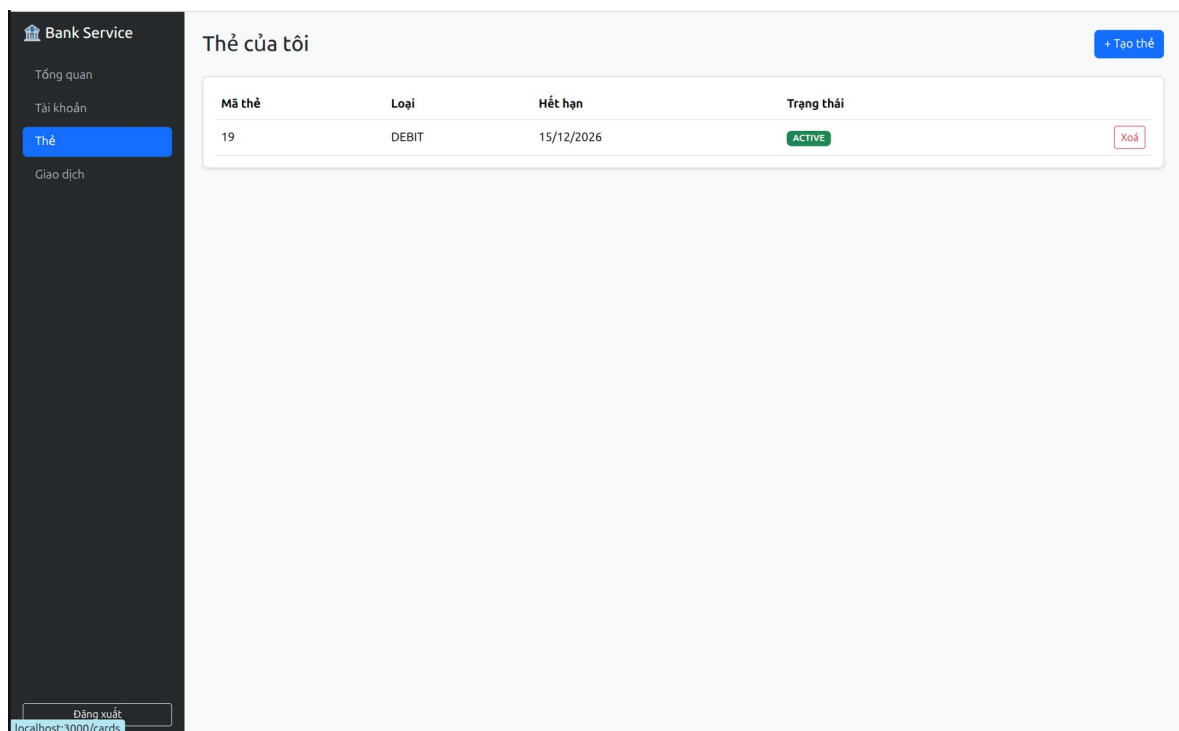
Thành phần	Địa chỉ
Giao diện web	http://localhost:3000
API bank-service	http://localhost:8080
Trang web có sẵn của ActiveMQ (xem hàng đợi)	http://localhost:8161 (đăng nhập admin / admin)
Tài khoản mẫu	a@test.com / Test@1234 (xem mục 4.6)

6.2. Giao diện

Giao diện có 6 trang: đăng nhập, đăng ký, tổng quan, tài khoản, thẻ và giao dịch. Lỗi từ backend (ví dụ không đủ số dư, thẻ hết hạn) được hiển thị ngay trên màn hình.



Hình 6.1: Trang tổng quan: thông tin tài khoản và số dư



Hình 6.2: Trang thẻ: danh sách thẻ và trạng thái

The screenshot shows a web application interface for a bank service. On the left is a dark sidebar with navigation links: "Bank Service", "Tổng quan", "Tài khoản", "Thẻ", "Giao dịch" (highlighted in blue), and "Đăng xuất". The main content area is titled "Giao dịch" and has a blue button in the top right corner showing "Số dư khả dụng: 9.500.000 đ". Below the title are three tabs: "Nạp tiền", "Rút tiền" (selected), and "Thanh toán". A red error message box states "Số dư khả dụng không đủ". Below this, there is a form with a "Số tiền (đ)" input field containing "100000000", a "Chọn thẻ" dropdown menu showing "Thẻ 19 - DEBIT - ACTIVE - hết hạn 15/12/2026", and a blue "Rút tiền" button.

Hình 6.3: Trang giao dịch: báo lỗi khi rút quá số dư khả dụng

6.3. Unit test

Unit test viết cho tầng Service bằng JUnit 5 và Mockito, Repository được giả lập nên không cần database. Mỗi API có ít nhất 1 test thành công và 1 test thất bại. Kết quả: **29/29 test đạt**.

```
./mvnw test
```

Bảng 6.2: Số lượng unit test

Lớp test	Số test	Nội dung chính
AccountServiceImplTest	8	Đăng ký, xem, sửa, xóa tài khoản; trùng email; còn thẻ
CardServiceImplTest	6	Xem, tạo, xóa thẻ; tài khoản không tồn tại; thẻ còn giao dịch PENDING
BalanceServiceImplTest	7	Xem số dư, nạp, rút; không đủ số dư; thẻ hết hạn
AuthServiceImplTest	2	Đăng nhập đúng; sai mật khẩu
PaymentServiceImplTest (bank)	2	Thanh toán thành công; payment-service lỗi → hoàn tiền
PaymentServiceImplTest (payment)	2	Gửi message; trùng paymentId

Lớp test	Số test	Nội dung chính
NotificationServiceImplTest	2	Ghi thông báo; message trùng
Tổng	29	

```

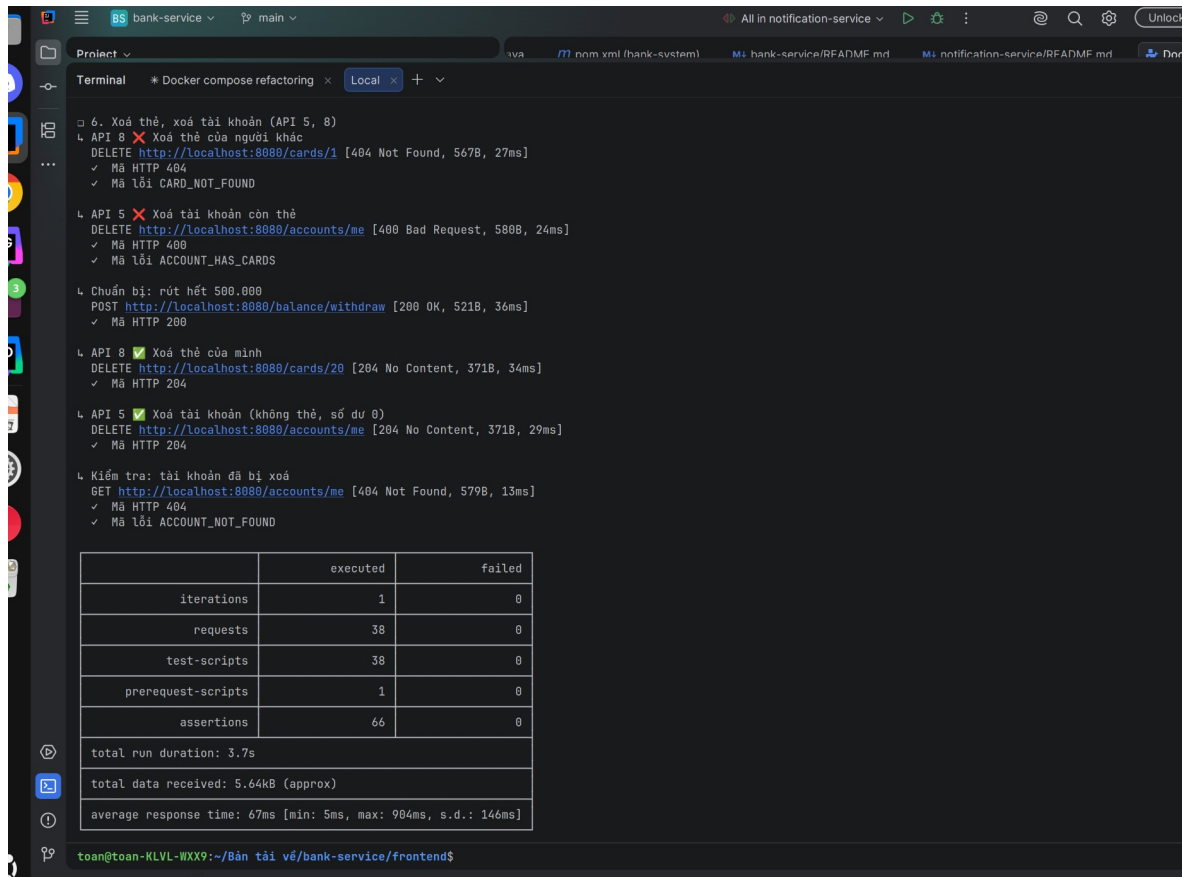
toan@toan-KLVL-WXX9: ~/Bản tải về/bank-service
[INFO] --- resources:3.3.1:resources (default-resources) @ notification-service ---
[INFO] Copying 1 resource from src/main/resources to target/classes
[INFO] Copying 0 resource from src/main/resources to target/classes
[INFO] --- compiler:3.13.0:compile (default-compile) @ notification-service ---
[INFO] Nothing to compile - all classes are up to date.
[INFO] --- resources:3.3.1:testResources (default-testResources) @ notification-service ---
[INFO] skip non existing resourceDirectory /home/toan/Bản tải về/bank-service/notification-service/src/test/resources
[INFO] --- compiler:3.13.0:testCompile (default-testCompile) @ notification-service ---
[INFO] Nothing to compile - all classes are up to date.
[INFO] --- surefire:3.5.2:test (default-test) @ notification-service ---
[INFO] Using auto detected provider org.apache.maven.surefire.junitplatform.JUnitPlatformProvider
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running com.bank.notification.service.service.impl.NotificationServiceImplTest
Mockito is currently self-attaching to enable the inline mock-maker. This will no longer work in future releases of the JDK. Please add Mockito as an agent to your build tool's documentation: https://javadoc.io/doc/org.mockito/mockito-core/latest/org/mockito/Mockito.html#0.3
OpenJDK 64-Bit Server VM warning: Sharing is only supported for boot loader classes because bootstrap classpath has been appended
WARNING: A Java agent has been loaded dynamically (/home/toan/.m2/repository/net/bytebuddy/byte-buddy-agent/1.15.11/byte-buddy-agent-1.15.11.jar)
WARNING: If a serviceability tool is in use, please run with -XX:+EnableDynamicAgentLoading to hide this warning
WARNING: If a serviceability tool is not in use, please run with -Djdk.instrument.traceUsage for more information
WARNING: Dynamic loading of agents will be disallowed by default in a future release
17:26:38.498 [main] INFO com.bank.notification.service.service.impl.NotificationServiceImpl -- Payment confirmed for paymentId: 44
17:26:38.581 [main] INFO com.bank.notification.service.service.impl.NotificationServiceImpl -- paymentId 44 đã thông báo trước đó, bỏ qua
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.317 s -- in com.bank.notification.service.service.impl.NotificationServiceImplTest
[INFO] Results:
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0
[INFO] -----
[INFO] Reactor Summary for bank-system 0.0.1-SNAPSHOT:
[INFO] bank-system ..... SUCCESS [ 0.002 s]
[INFO] bank-service ..... SUCCESS [ 7.751 s]
[INFO] payment-service ..... SUCCESS [ 2.232 s]
[INFO] notification-service ..... SUCCESS [ 1.882 s]
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 12.232 s
[INFO] Finished at: 2026-10-05T17:26:38+07:00
[INFO] -----
toan@toan-KLVL-WXX9: ~/Bản tải về/bank-service$

```

Hình 6.4: Kết quả chạy unit test: cả 3 module đều SUCCESS

6.4. Kiểm thử API bằng Postman

Bộ Postman collection ([postman/bank-service.postman_collection.json](#)) gồm 38 request, phủ đủ 12 API, mỗi API có ít nhất 1 trường hợp thành công và 1 trường hợp thất bại. Collection tự lưu token và mã thẻ vào biến, tự tạo tài khoản mới ở đầu và xóa ở cuối nên chạy lại nhiều lần vẫn đúng. Collection được chạy bằng Newman (bản dòng lệnh của Postman). Kết quả: **38/38 request đạt**, 66/66 kiểm tra đúng.



Hình 6.5: Kết quả chạy Postman collection bằng Newman: 38 request, 66 kiểm tra, 0 lỗi

6.5. Kiểm thử khi một thành phần bị tắt

Bảng 6.3: Kết quả khi tắt từng thành phần

Tắt	Kết quả
Redis	Các API vẫn chạy, đọc thẳng database; log ghi cảnh báo
payment-service	Thanh toán trả 503, tiền được hoàn đủ; các API khác bình thường
notification-service	Thanh toán vẫn thành công; message nằm chờ trong payment.queue , bật lại thì tự xử lý
postgres-payment	bank-service và notification-service vẫn hoạt động

The screenshot shows the ActiveMQ web console interface. At the top, there's a navigation bar with links: Home, Queues, Topics, Subscribers, Connections, Network, Scheduled, and Send. Below this, there's a search bar for Queue Name and a Filter button. The main content area displays a table of queues. The table has columns: Name, Number Of Pending Messages, Number Of Consumers, Messages Enqueued, Messages Dequeued, Views, and Operations. The first row shows 'payment.queue' with 2 pending messages, 0 consumers, 3 enqueued messages, and 1 dequeued message. To the right of the table, there are buttons for 'Browse Active Consumers', 'Active Producers', and 'Send To Purge Delete Pause'. On the right side of the console, there's a sidebar with links for Queue Views (Graph, XML), Topic Views (XML), Subscribers Views (XML), and Useful Links (Documentation, FAQ, Downloads, Forums). The footer of the console shows 'Copyright 2005-2023 The Apache Software Foundation'.

Name	Number Of Pending Messages	Number Of Consumers	Messages Enqueued	Messages Dequeued	Views	Operations
payment.queue	2	0	3	1	Browse Active Consumers Active Producers	Send To Purge Delete Pause

Hình 6.6: *payment.queue* khi *notification-service* tắt: 2 message đang chờ, 0 consumer

CHƯƠNG 7. KẾT LUẬN

7.1. Kết quả đạt được

- Hoàn thành 12 API theo đề bài, tổ chức 3 tầng Controller – Service – Repository, đảm bảo 3 ràng buộc nghiệp vụ.
- Lấy `accountId` từ JWT; cache Redis cho tài khoản và số dư, TTL 10 phút.
- `payment-service` và `notification-service` giao tiếp qua ActiveMQ, có chống xử lý trùng.
- Toàn bộ 9 container chạy bằng Docker Compose; 29/29 unit test và 38/38 request Postman đạt.
- Có giao diện web React cho người dùng thao tác.

7.2. Hạn chế

- Thông báo mới dừng ở mức ghi log, chưa gửi email hoặc SMS thật.
- Thẻ CREDIT và DEBIT đang xử lý giống nhau, chưa có hạn mức tín dụng.
- Token lưu ở `localStorage` của trình duyệt; sản phẩm thật nên dùng cookie `HttpOnly`.
- Chưa có xem lịch sử giao dịch, chuyển khoản và phân quyền quản trị (ngoài phạm vi đề bài).

7.3. Hướng phát triển

- Gửi email xác nhận thanh toán từ `notification-service`.
- Thêm API xem lịch sử giao dịch và chuyển khoản giữa các tài khoản.
- Thêm vai trò quản trị viên và refresh token.